

# Assignment 04 - Distributed Programming

## Problem Analysis

This assignment moves from local concurrency to distributed programming. The implemented systems are a Smart Home Alarm based on Apache Pekko Cluster, a Tic-Tac-Toe game based on Java RMI, and a distributed critical-section middleware based on RabbitMQ.

## Exercise 1 - Distributed Smart Home Alarm

Pekko Cluster nodes can host control, keypad, and sensor roles. Actors communicate only through messages; the control panel is discovered through Receptionist and all protocol messages are serializable.

```
RECOVERY -- valid PIN --> DISARMED
DISARMED -- valid PIN arm --> EXIT_DELAY -- timeout --> ARMED
ARMED -- active-zone sensor --> ENTRY_DELAY -- timeout --> ALARM
```

```
.\mvnw.cmd exec:java "-Dexec.args=control 25520"
.\mvnw.cmd exec:java "-Dexec.args=keypad 25521 demo"
.\mvnw.cmd exec:java "-Dexec.args=sensor 25522 front-door perimeter DOOR demo"
```

## Exercise 2 - Distributed Tic-Tac-Toe With Java RMI

The RMI system uses TicTacToeHub, TicTacToeGame, and TicTacToeListener. Game methods are synchronized on the server and callbacks are removed on RemoteException.

```
.\mvnw.cmd exec:java "-Dexec.mainClass=pcd.ass04.ttt.TicTacToeServer" "-Dexec.args=1099"
.\mvnw.cmd exec:java "-Dexec.mainClass=pcd.ass04.ttt.TicTacToeClient" "-Dexec.args=localhost 1099 create game1 Alice"
.\mvnw.cmd exec:java "-Dexec.mainClass=pcd.ass04.ttt.TicTacToeClient" "-Dexec.args=localhost 1099 join game1 Bob"
```

## Exercise 3 - Distributed Critical Sections With RabbitMQ

The middleware exposes DistributedLock. A process only needs a lock name and process id; it does not know the other participants. RabbitMQ hosts request/release queues and each process owns an exclusive reply queue.

```
on request:
  if no current holder -> grant immediately
  else enqueue request

on release:
  if queue empty -> lock becomes free
  else dequeue first waiting process and grant it
```

```
docker compose up -d rabbitmq
.\mvnw.cmd exec:java "-Dexec.mainClass=pcd.ass04.cs.CriticalSectionDemo"
```

```
p2:enter,p2:exit,p3:enter,p3:exit,p1:enter,p1:exit
```

## Verification

```
.\mvnw.cmd clean test
Tests run: 8, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS
```

```
.\mvnw.cmd package  
BUILD SUCCESS
```